Exception Handling

- Exception handling is a method used to handle errors that occur during program execution.
- It prevents the program from stopping suddenly and allows controlled execution.

Examples of Exceptions

- Division by zero
- File not found
- Invalid input
- Array index out of range

Need for Exception Handling

Exception handling is required to:

- Prevent program crash
- Maintain normal flow of execution
- Display meaningful error messages
- Improve reliability of program

Structure of Exception Handling

Most programming languages use the following blocks:

1. **Try Block -** Contains code that may generate an exception.
2. **Catch Block -** Handles the exception when an error occurs.
3. **Finally Block -** Executes whether an exception occurs or not.

Working of Exception Handling

1. The program executes code inside the try block.
2. If an error occurs, an exception is generated.
3. Control moves to catch blocks.
4. Exceptions are handled properly.
5. Finally the block executes at the end.

Types of Exceptions

1. Compile-time Exception - Errors found during compilation.

Example: Syntax errors.

2. Run-time Exception - Errors found during execution.

Example: Divide by zero.

3. Logical Error

The program runs but produces wrong output.

Example: Wrong formula in calculation.

Exception Handling in Java

Java uses `try`, `catch`, `finally`, `throw`, and `throws`.

Example

```
try {
   int a = 10/0;
}
catch(Exception e) {
   System.out.println("Error");
}
finally {
   System.out.println("Done");
}
```

Advantages of Exception Handling

- Prevents sudden program termination
- Improves program stability
- Makes debugging easier
- Gives meaningful error messages
- Improves user friendliness

Event Handling

Event handling is the process of detecting and responding to events that occur during program execution.
An event can be generated by user actions like

Example of Events

- Mouse click
- Keyboard press
- Button click
- Window closing

Components of Event Handling

1. Event - An event is an action or occurrence detected by the program.

Examples: Clicking a button, Pressing a key

2. Event Source -The object that generates the event.

Examples: Button, Mouse, Keyboard

3. Event Handler - A function or method that responds to the event.

Example: A function that opens a file when a button is clicked.

4. Event Listener - Listens for events and sends them to the event handler.

Working of Event Handling

1. User performs an action
2. Event is generated
3. Event listener detects the event
4. Control moves to event handler
5. Appropriate action is performed

Example of Event Handling in Java

button.addActionListener(this);

Explanation

- Button generates an event
- Listener detects the event
- Handler performs required action

Advantages of Event Handling

- Makes programs interactive
- Improves user experience
- Supports GUI applications
- Allows efficient user interaction

Statement Level Concurrency

Statement level concurrency means executing multiple statements of a program at the same time. It improves program performance and reduces execution time.

Normally, program statements execute one after another in sequence.
In statement level concurrency, independent statements are executed simultaneously using parallel processing.

Working of Statement Level Concurrency

- The compiler checks the program for independent statements.
- Independent statements are executed simultaneously.
- Dependent statements execute sequentially.

Processors use multiple cores and pipelining for parallel execution.

Types of Dependencies

1. Data Dependency - Occurs when one statement depends on another statement's result.

Example

A = B + C
D = A + E

Here, the second statement depends on the value of `A`, so both cannot execute together.

2. Control Dependency - Execution depends on a condition.

Example

```
if (A > 0)
  B = C + D
```

The statement executes only if the condition is true.

Techniques Used

Compilers and processors use different techniques to achieve concurrency.

Compiler Techniques

- Instruction reordering
- Parallel execution

Processor Techniques

- Pipelining
- Multiple cores

Sequential vs Concurrent Execution

Sequential Execution

Step 1 → Statement 1

Step 2 → Statement 2

Step 3 → Statement 3

Statements execute one after another.

Concurrent Execution

Step 1 → Statement 1 & Statement 2

Step 2 → Statement 3

Multiple statements execute together.

Advantages of Statement Level Concurrency

- Improves execution speed
- Efficient CPU utilization
- Reduces execution time
- Increases overall performance

Difference Between Sequential and Concurrent Execution

S.No	Sequential Execution	Concurrent Execution
1	Statements execute one by one	Statements execute simultaneously
2	Slower execution	Faster execution
3	Less CPU utilization	Better CPU utilization
4	Simple execution	Complex execution

Design Issues for Object-Oriented Programming (OOP) Languages

Design issues in OOP languages are the important decisions taken while designing features like classes, objects, inheritance, and polymorphism.

Important Design Issues in OOP Languages

1. Object and Class Design

Objects represent real-world entities and classes act as blueprints.

- Organizes program properly
- Combines data and methods together

---

2. Encapsulation

Encapsulation means hiding data inside a class using access specifiers.

Access Specifiers

- Public
- Private
- Protected

Advantages

- Protects data
- Improves security

3. Inheritance

Inheritance allows one class to acquire properties of another class.

Types

- Single inheritance
- Multiple inheritance
- Multilevel inheritance

Advantages

- Reduces code duplication
- Supports code reuse

---

4. Polymorphism

Polymorphism allows the same function to behave differently in different situations.

Types

- Method overloading
- Method overriding

Advantages

- Improves flexibility
- Supports dynamic behavior

---

5. Message Passing

Objects communicate with each other through method calls.

Advantages

- Improves interaction between objects
- Supports modular programming

---

6. Exception Handling

Exception handling manages runtime errors without stopping the program.

Advantages

- Improves stability
- Prevents program crash

Semaphores

- A semaphore is a synchronization tool used in operating systems and concurrent programming to control access to shared resources by multiple processes or threads.
- When multiple processes access the same resource at the same time, problems like data inconsistency may occur.
- Semaphores help manage and coordinate process execution safely.

Working of Semaphore

A semaphore uses an integer variable and two operations:

1. Wait Operation (P)

- Decreases semaphore value by 1
- If value becomes negative, process waits

2. Signal Operation (V)

- Increases semaphore value by 1
- Wakes up waiting process

Types of Semaphores

1. Binary Semaphore

- Value can be only 0 or 1
- Used for mutual exclusion

2. Counting Semaphore

- Value can be greater than 1
- Controls multiple resource instances

Example of Semaphore

wait(S);

critical section

signal(S);

Explanation

- `wait(S)` checks resource availability
- Critical section executes safely
- `signal(S)` releases resource

Advantages of Semaphores

- Prevents race conditions
- Provides process synchronization
- Efficient resource sharing
- Supports concurrent execution

Threads

A thread is the smallest unit of execution within a process. Threads are used to perform multiple tasks at the same time within a process.

Features of Threads

- Threads share the same memory and resources of a process
- Each thread has its own execution path
- Threads execute independently

Types of Threads

1. User-Level Threads

- Managed by user-level libraries
- Faster thread creation and management

---

2. Kernel-Level Threads

- Managed directly by operating system kernel
- Better multitasking support

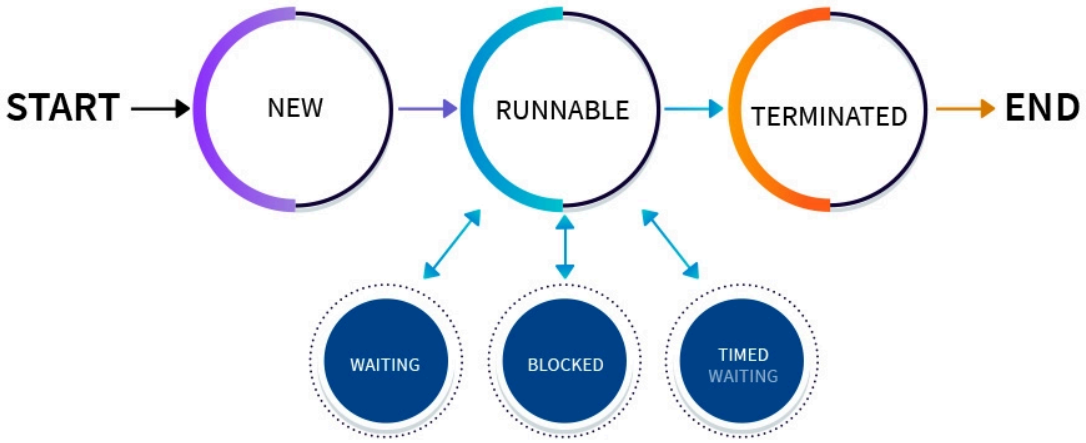
Multithreading

Multithreading means executing multiple threads simultaneously in a program.

Example

- Web browsers running multiple tabs
- Downloading file while playing music

Life Cycle of a Thread



Advantages of Threads

- Faster execution
- Better CPU utilization
- Improved responsiveness
- Efficient resource sharing